

Logic Programming & Prolog — Complete Finals Guide

A ground-up walkthrough, simple words first, tricky stuff last.

PART 1: THE LOGIC FOUNDATIONS (before Prolog even makes sense)

1.1 Proposition

A **proposition** is a statement that is either **TRUE** or **FALSE** — never both, never neither.

- "Dhaka is in Bangladesh." → proposition (True)
- " $2 + 2 = 5$ " → proposition (False)
- "What time is it?" → **NOT** a proposition (it's a question, no truth value)
- " $x + 1 = 5$ " → **NOT** a proposition by itself (x is unknown — becomes one only once x is fixed)

1.2 Compound Proposition

A proposition built by joining two or more simple propositions using **connectives** (AND, OR, NOT, IMPLIES, IFF).

Example: "It is raining AND I have an umbrella" = $P \wedge Q$

1.3 The Connectives (with Truth Tables)

Let P and Q be propositions.

Conjunction (AND) — symbol: $\wedge$ True only when BOTH are true.

P	Q	$P \wedge Q$
T	T	T
T	F	F

F T F

F F F

Disjunction (OR) — symbol: $\vee$ True when AT LEAST ONE is true.

P	Q	$P \vee Q$
---	---	------------

T	T	T
---	---	---

T	F	T
---	---	---

F	T	T
---	---	---

F	F	F
---	---	---

Negation (NOT) — symbol: $\neg$

P	$\neg P$
---	----------

T	F
---	---

F	T
---	---

Implication (IF...THEN) — symbol: $\rightarrow$, read "**P implies Q**" Only FALSE when a true thing leads to a false thing. Otherwise true. (Think of it as a promise: "If P then Q" is only broken when P happens but Q doesn't.)

P	Q	$P \rightarrow Q$
---	---	-------------------

T	T	T
---	---	---

T	F	F ← only false case
---	---	---------------------

F	T	T
---	---	---

F	F	T
---	---	---

Biconditional (IF AND ONLY IF / IFF) — symbol: $\leftrightarrow$ True when P and Q have the SAME truth value (both true or both false).

P	Q	$P \leftrightarrow Q$
---	---	-----------------------

T	T	T
---	---	---

T F F

F T F

F F T

Trick to remember: Biconditional = $(P \rightarrow Q) \wedge (Q \rightarrow P)$. It's implication working BOTH ways at once.

1.4 Terminology: Hypothesis, Conclusion, Implies, Synonyms

For $P \rightarrow Q$:

- P = **hypothesis** (also called: antecedent, premise, "if" part)
- Q = **conclusion** (also called: consequent, "then" part)

Synonyms/phrasings of "P implies Q" ($P \rightarrow Q$) — these ALL mean the exact same thing (very common trick question):

- "If P, then Q"
- "P is sufficient for Q"
- "Q is necessary for P"
- "Q if P"
- "P only if Q"
- "Q follows from P"
- "Q whenever P"

Synonyms for "iff" ($P \leftrightarrow Q$):

- "P if and only if Q"
- "P is necessary and sufficient for Q"
- "P exactly when Q"

1.5 Equivalence — the Bridge Between Conjunction and Implication

Two very important **logical equivalences** connect the connectives together:

1. **Implication rewritten using OR/NOT:** $P \rightarrow Q \equiv \neg P \vee Q$ (Trick: "if P then Q" is the same as "either not-P, or Q")
2. **Biconditional rewritten using AND/Implication:** $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$

This is why the guide title calls **equivalence "the bridge"**: it lets you convert AND / OR statements into IMPLICATION statements and back — which is exactly what Prolog's rule engine relies on internally (Prolog rules are all built from implication-like structures).

Quick equivalence table (very examinable):

Statement	Equivalent to
$P \rightarrow Q$	$\neg P \vee Q$
$\neg(P \wedge Q)$	$\neg P \vee \neg Q$ (De Morgan's)
$\neg(P \vee Q)$	$\neg P \wedge \neg Q$ (De Morgan's)
$P \leftrightarrow Q$	$(P \rightarrow Q) \wedge (Q \rightarrow P)$
$P \rightarrow Q$	$\neg Q \rightarrow \neg P$ (contrapositive)

PART 2: PREDICATES, QUANTIFIERS, FIRST ORDER LOGIC

2.1 Predicate

A proposition needs a fixed value to be true/false. A **predicate** is like a "template" proposition with a variable slot, that becomes true/false once you plug something in.

Example: `Student(x)` = "x is a student"

- `Student(Rafi)` → could be True
- `Student(Table)` → False

Predicates let you express general patterns instead of one-off facts.

2.2 Quantifiers

Quantifiers tell you **how many** things a predicate applies to.

Universal Quantifier — $\forall$ ("for all") $\forall x \ P(x)$ = "P(x) is true for EVERY x"

- Example: $\forall x \ (\text{Human}(x) \rightarrow \text{Mortal}(x))$ = "All humans are mortal"

Existential Quantifier — $\exists$ ("there exists") $\exists x \ P(x)$ = "P(x) is true for AT LEAST ONE x"

- Example: $\exists x \text{ Student}(x) \wedge \text{Smart}(x)$ = "There exists a student who is smart"

Tricky part — negating quantifiers (flips the quantifier AND negates inside):

Statement	Negation
$\neg \forall x P(x)$	$\exists x \neg P(x)$
$\neg \exists x P(x)$	$\forall x \neg P(x)$

Mnemonic: "Not all" becomes "there exists one that doesn't"; "none exist" becomes "all fail".

2.3 First Order Logic (FOL)

FOL = Propositional logic + **predicates + quantifiers + variables/objects**. It lets you talk about objects, their properties, and relationships between them — not just fixed True/False statements.

FOL sentence example: $\forall x (\text{Bird}(x) \wedge \neg \text{Penguin}(x) \rightarrow \text{CanFly}(x))$ = "For all x, if x is a bird and x is not a penguin, then x can fly."

This is **exactly the mathematical form that Prolog rules are built on** — a Prolog rule is basically an FOL implication written backward (see Part 3).

PART 3: LOGIC PROGRAMMING & PROLOG BASICS

3.1 What is Logic Programming?

A programming paradigm where you don't tell the computer *how* to solve a problem step-by-step (like in C/Python). Instead, you give it:

1. **Facts** (things that are true)
2. **Rules** (relationships / logic, using implication)

...and then you **ask it questions (queries)**. The computer uses **inference** (logical deduction) to figure out the answer.

Prolog ("PROgramming in LOGic") is the most famous logic programming language.

SWI-Prolog is simply one specific, free, widely-used implementation/software of Prolog (like how Python and CPython relate — Prolog is the language, SWI-Prolog is the tool you run it in).

3.2 Knowledge Base (KB)

The **Knowledge Base** is the full collection of facts + rules you've written in your `.pl` file. It's Prolog's "memory" — everything it knows.

3.3 Facts

A **fact** is a statement declared as unconditionally true. No conditions, no "if".

```
male(ram).  
female(sita).  
parent(ram, luv).  
likes(ram, mango).
```

Read `parent(ram, luv).` as: "ram is a parent of luv" — always true, no question asked.

"Element related → only facts are written" — meaning: when you're just declaring basic elements/objects and their direct properties or relationships (not general logic patterns), you use plain facts, not rules.

3.4 Rules

A **rule** states something is true **only if some condition(s) hold**. This is literally implication ($\rightarrow$) written in Prolog syntax, but BACKWARDS (conclusion first, then condition).

`:-` means "implies" / "if" — read RIGHT TO LEFT ("Head is true IF Body is true")

```
father(X, Y) :- parent(X, Y), male(X).
```

Read as: "X is the father of Y, **IF** X is a parent of Y **AND** X is male."

`,` means AND — every comma-separated goal in the body must be true.

```
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

"X is grandparent of Z if X is parent of Y AND Y is parent of Z."

You can also use `;` for OR in the body (less common, often avoided for clarity):

```
sibling(X, Y) :- parent(Z, X), parent(Z, Y), (male(X) ; female(X)).
```

3.5 Horn Clause & Headless Horn Clause

A **Horn Clause** is a logical clause with **at most one positive (head) conclusion**. Prolog facts and rules are ALL Horn clauses.

General form: **Head** :- **Body**.

- Facts = Horn clause with an empty body (Head :- true.), written simply as **Head**.
- Rules = Horn clause with Head + Body

Headless Horn Clause = a Horn clause with **NO head**, only a body. Written as:

:- Body.

This represents a **query** or a **goal/directive** — "prove that Body is true" — there's no conclusion being defined, you're just asking Prolog to evaluate/prove something. This is exactly what a **query** is structurally.

Trick connection: Query = Headless Horn Clause. Fact = Bodyless Horn Clause (body is empty/true). Rule = Full Horn clause (both head and body).

3.6 The 3 Steps of Prolog Programming

1. **Knowledge Base** — define your facts.
2. **Rule Definition** — define your rules (relationships using :- and ,).
3. **Query** — ask questions against the KB + rules.

% STEP 1: Knowledge Base (facts)

male(ram).

male(luv).

parent(ram, luv).

% STEP 2: Rule Definition

father(X, Y) :- parent(X, Y), male(X).

% STEP 3: Query (typed at the ?- prompt)

?- father(ram, luv).

% Output: true.

3.7 Empty Clause

In resolution-based inference (which is how Prolog proves things internally), an **empty clause** (written `[]` or `□`) means that all conditions have been successfully resolved/cancelled out — it's the signal that the proof is **COMPLETE and SUCCESSFUL** (or in refutation proofs, that a contradiction was found, proving the original goal).

Simply put: reaching the empty clause = "nothing left to prove" = Prolog says **true**.

3.8 Trace

trace is a built-in Prolog command that turns on the **debugger**. Once on, Prolog shows you every step it takes (Call, Exit, Redo, Fail) while trying to solve a query — very useful to see exactly HOW it searched the knowledge base.

```
?- trace.
```

```
?- father(ram, luv).
```

```
Call: father(ram, luv)
```

```
Call: parent(ram, luv)
```

```
Exit: parent(ram, luv)
```

```
Call: male(ram)
```

```
Exit: male(ram)
```

```
Exit: father(ram, luv)
```

```
true.
```

Turn it off with **notrace**.

PART 4: INFERENCE & SOLVING QUERIES (the exam-favorite part)

4.1 What is the "Inference Problem"?

Given a Knowledge Base (facts + rules), and a Query, **can the system logically DERIVE (infer) whether the query is true**, using only what's in the KB? This is the core job Prolog does every time you run a query — it's called **inference** or **resolution**.

4.2 How Prolog Solves a Query (step by step, like a database lookup)

Think of facts as **rows in a database table**, and rules as **derived views**. When you query:

```
% KB
```

```
parent(ram, luv).
```

```
parent(ram, kush).
```

```
parent(dashrath, ram).
```

```
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```



```
% Query
?- grandparent(dashrath, luv).
```

Prolog's inference steps:

1. Match `grandparent(dashrath, luv)` against the rule head `grandparent(X, Z)` → binds `X=dashrath, Z=luv`.
2. Now must prove the body: `parent(dashrath, Y), parent(Y, luv)`
3. Search KB for `parent(dashrath, Y)` → finds `parent(dashrath, ram)` → binds `Y=ram`.
4. Now must prove `parent(ram, luv)` → search KB → **found as a fact!**
5. Both conditions of the body proved → empty clause reached → **true**.

This process — matching facts, substituting variables, chaining rules — **is exactly how you'd answer a database query using inference** (e.g., "find all grandchildren of X" is like a SQL JOIN across two tables, but Prolog does it via unification + backtracking instead of joins).

Database-style query (asking "who"):

```
?- grandparent(dashrath, Who).
Who = luv ;
Who = kush.
```

Prolog searches ALL possible matches (like `SELECT` returning multiple rows), and `;` asks for the next solution (backtracking).

4.3 Factorial — the Classic Recursive Example

```
factorial(0, 1).                % Base fact: 0! = 1
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.
```

Read the rule as: "F is factorial of N, IF N > 0, AND N1 = N-1, AND F1 is factorial(N1), AND F = N × F1."

Trace for `factorial(3, F)`:

```
factorial(3,F) → 3>0 → N1=2 → factorial(2,F1)
factorial(2,F1) → 2>0 → N1=1 → factorial(1,F1')
```

$\text{factorial}(1, F1') \rightarrow 1 > 0 \rightarrow N1=0 \rightarrow \text{factorial}(0, F1'')$
 $\text{factorial}(0, F1'') = 1$ (base fact hits — empty clause reached here)
 back-substitute: $F1' = 1 * 1 = 1$
 $F1 = 2 * 1 = 2$
 $F = 3 * 2 = 6$

`?- factorial(3, F). → F = 6.`

Trickiest part students miss: the base case (`factorial(0, 1)`) is a **fact**, and it's what stops the recursion — without it, Prolog would recurse forever trying to prove $N > 0$. This is identical logically to how the **empty clause** terminates a resolution proof.

PART 5: QUICK-FIRE REVISION TABLE

Term	One-line meaning
Proposition	Statement that is only True or False
Compound proposition	Multiple propositions joined by connectives
Conjunction ($\wedge$)	AND — true only if both true
Disjunction ($\vee$)	OR — true if at least one true
Implication ($\rightarrow$)	IF...THEN — false only when true $\rightarrow$ false
Biconditional ($\leftrightarrow$)	IFF — true when both sides match
Hypothesis	The "if" part (P) of $P \rightarrow Q$
Equivalence	Statements that always have the same truth value; bridges AND/OR $\leftrightarrow$ implication
Predicate	A statement template with variables, e.g. <code>Student(x)</code>
Quantifier	$\forall$ (for all) / $\exists$ (there exists)
First Order Logic	Propositional logic + predicates + quantifiers
Knowledge Base	All facts + rules in your Prolog program
Fact	Unconditional truth, e.g. <code>male(ram).</code>

Rule	Conditional truth using <code>:-</code> and <code>,</code>
<code>:-</code>	Means "implies" / "if", read right-to-left
<code>,</code>	Means AND (inside a rule body)
Horn clause	Clause with at most one head; facts & rules are Horn clauses
Headless Horn clause	Horn clause with no head = a query/goal (<code>:- Body.</code>)
Empty clause	Signals proof is complete (nothing left to resolve)
Query	A question asked to the KB, <code>?- goal.</code>
trace.	Turns on step-by-step debugging
Inference	Logical process of deriving new true facts from KB

PART 6: THE TRICKIEST EXAM PATTERNS

Pattern A — "Which of these are equivalent?" using De Morgan's + implication rewrite.
 Always convert everything to $\wedge/\vee/\neg$ form first using $P \rightarrow Q \equiv \neg P \vee Q$, then compare.

Pattern B — writing FOL for English sentences. "Every student who studies passes" $\rightarrow \forall x (\text{Student}(x) \wedge \text{Studies}(x) \rightarrow \text{Passes}(x))$ Watch: "every ___ who ___" almost always becomes $\forall x (A(x) \wedge B(x) \rightarrow C(x))$, NOT $\forall x (A(x) \rightarrow (B(x) \wedge C(x)))$ — a very common mistake.

Pattern C — converting an FOL rule into Prolog. $\forall x (\text{Bird}(x) \wedge \neg \text{Penguin}(x) \rightarrow \text{Flies}(x))$ becomes:

`flies(X) :- bird(X), \+ penguin(X).`

(`\+` = "not provable" — Prolog's negation)

Pattern D — multiple rules for the same predicate (OR logic via multiple clauses).

`grandparent(X,Z) :- father(X,Y), parent(Y,Z).`
`grandparent(X,Z) :- mother(X,Y), parent(Y,Z).`

Two separate clauses for the same head = logical OR between them (Prolog tries each in order, backtracking if one fails).

Pattern E — recursive rules always need a base fact, or Prolog loops forever (see factorial above). Always check: does the recursive rule have a non-recursive fact to bottom out on?

Pattern F — tracing backtracking with ;. When a query has multiple possible answers, typing ; after the first result asks Prolog to backtrack and find the next one — this is Prolog systematically trying alternate ways to reach an empty clause.

PART 7: BACKTRACKING (in depth)

Backtracking = when Prolog's current path fails (or you ask for more answers with ;), it goes back to the last choice point and tries the NEXT alternative instead.

```
fruit(apple).  
fruit(mango).  
fruit(banana).
```

```
?- fruit(X).  
X = apple ;  
X = mango ;  
X = banana.
```

What actually happens internally:

1. Prolog tries `fruit(apple)` → matches first fact → `X=apple`, returns it.
2. You type `;` → Prolog "un-does" that binding, goes back to where it left off in the fact list, tries the NEXT clause `fruit(mango)` → `X=mango`.
3. Repeats until no clauses are left → then `false`.

Backtracking with rules (multi-goal bodies) — the trickiest kind:

```
parent(ram, luv). parent(ram, kush). parent(dashrath, ram).  
grandparent(X,Z) :- parent(X,Y), parent(Y,Z).
```

```
?- grandparent(dashrath, W).
```

Step by step:

- X=dashrath. Find `parent(dashrath, Y)` → only match Y=ram.
- Now try `parent(ram, Z)` → first match Z=luv → **answer 1: W=luv**
- User asks for more (;) → Prolog backtracks to the LAST choice point, which was `parent(ram, Z)` (not `parent(dashrath, Y)`, since that only had one option) → tries next match Z=kush → **answer 2: W=kush**
- Backtrack again → no more `parent(ram, _)` facts, AND no more `parent(dashrath, _)` alternatives either → **false**.

Key exam point: backtracking always retries the MOST RECENT unexplored choice point first (like a stack) — this is called depth-first search with backtracking, and it's exactly how Prolog explores the whole knowledge base.

Cut (!) — stops backtracking at that point (worth knowing alongside backtracking, often tested together):

```
first_fruit(X) :- fruit(X), !.
```

Once **!** is passed, Prolog will NOT backtrack past it — it commits to the first solution found.

PART 8: LOGICAL OPERATORS, EQUIVALENCE & EQUALITY — FULL LAWS

8.1 Complete List of Logical Equivalence Laws

Law name	Rule
Identity	$P \wedge T \equiv P ; P \vee F \equiv P$
Domination	$P \vee T \equiv T ; P \wedge F \equiv F$
Idempotent	$P \wedge P \equiv P ; P \vee P \equiv P$
Double negation	$\neg(\neg P) \equiv P$
Commutative	$P \wedge Q \equiv Q \wedge P ; P \vee Q \equiv Q \vee P$
Associative	$(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$
Distributive	$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$

De Morgan's $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$; $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

Absorption $P \vee (P \wedge Q) \equiv P$; $P \wedge (P \vee Q) \equiv P$

Negation $P \vee \neg P \equiv T$; $P \wedge \neg P \equiv F$

Implication rewrite $P \rightarrow Q \equiv \neg P \vee Q$

Biconditional rewrite $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$

De Morgan's Law explained simply: "Breaking a NOT over an AND/OR flips the connective."

- $\neg(\text{Rain} \wedge \text{Cold}) = \text{"It's NOT the case that it's raining AND cold"} \equiv \neg\text{Rain} \vee \neg\text{Cold} = \text{"either it's not raining, or it's not cold (or both)"}$

8.2 Equality (= in logic vs Prolog)

In pure logic, $=$ states two terms denote the same object.

In **Prolog**, $=$ attempts **unification** — it tries to make both sides identical (possibly by binding variables), and succeeds/fails accordingly:

```
?- X = 5.      % X unifies with 5 → X = 5.
?- 5 = 5.      % true.
?- 5 = 6.      % false.
?- father(ram,luv) = father(X,Y).  % X=ram, Y=luv (structural match)
```

This is different from $==$ (strict equality, no binding) and $=:$ (arithmetic equality, evaluates both sides as numbers).

8.3 Converse, Inverse, Contrapositive of $P \rightarrow Q$

Given the **original**: $P \rightarrow Q$

Name	Form	Logically equal to original?
Original	$P \rightarrow Q$	—
Converse	$Q \rightarrow P$	 No
Inverse	$\neg P \rightarrow \neg Q$	 No

Contrapositive $\neg Q \rightarrow \neg P$  YES — always equivalent

Memory trick: Converse and Inverse are logically equivalent to EACH OTHER (they're each other's contrapositive), but NOT to the original. Only the Contrapositive matches the original.

Example: "If it rains, the ground is wet." ($P \rightarrow Q$)

- Converse: "If the ground is wet, it rained." (not necessarily true — could be a sprinkler!)
- Inverse: "If it doesn't rain, the ground isn't wet." (also not necessarily true — same sprinkler problem)
- Contrapositive: "If the ground isn't wet, it didn't rain." (MUST be true if original is true)

PART 9: WHY PROPOSITIONAL LOGIC ISN'T ENOUGH

"Propositional logic can't say..." — lack of expressive power

Propositional logic treats sentences as whole, atomic blocks — it CANNOT look inside a sentence to talk about individual objects, properties, or "for all / some" patterns.

Classic example that breaks propositional logic:

- "All men are mortal."
- "Socrates is a man."
- Therefore, "Socrates is mortal."

In propositional logic, you'd have to write this as three totally unrelated atomic symbols: P = "All men are mortal", Q = "Socrates is a man", R = "Socrates is mortal"

There is **no logical connection** P , Q can be combined to prove R — propositional logic has no way to express that "men" and "Socrates" are related, because it can't see INSIDE the sentences to the objects and categories.

This is exactly why we need **predicate logic / FOL** — it can express: $\forall x (Man(x) \rightarrow Mortal(x)), Man(Socrates) \vdash Mortal(Socrates)$ — now the internal structure (objects, categories, "all") is visible and usable.

This is the standard exam answer for "give an example of something propositional logic cannot express": anything involving "all", "some", relationships between specific objects, or shared internal structure across different statements.

PART 10: PREDICATE LOGIC — DOMAINS, COMPLETE DEFINITIONS, QUANTIFIER RULES

10.1 Domain

The **domain** (or "universe of discourse") is the set of objects a quantifier ranges over. You **MUST** know the domain to correctly interpret $\forall$ / $\exists$ statements.

Example — Domain: Birds $\forall x (Bird(x) \rightarrow CanFly(x))$ — if the domain is restricted to just "birds", this simplifies to $\forall x CanFly(x)$, since $Bird(x)$ is true for every x already.

Example — Domain: Animals (broader) $\forall x (Animal(x) \wedge Bird(x) \rightarrow CanFly(x))$ — here we can't drop $Bird(x)$ because the domain now includes non-birds (cats, fish, etc.) too, so we must explicitly filter down to birds first.

Key exam trap: the SAME English sentence needs a DIFFERENT FOL formula depending on the stated domain. Always check what the domain is before writing quantified statements.

10.2 Complete Predicate Definition

A predicate is "completely defined" when you've specified its truth value for **every single element in the domain** — no ambiguity left. In Prolog, this means covering every case with facts/rules (including a base/default case), so a query never falls through with no matching clause.

10.3 Universal Quantifier ($\forall$) — recap + rule form

$\forall x P(x)$ — true if $P(x)$ holds for EVERY x in the domain. To DISPROVE it, you only need ONE counter-example.

10.4 Existential Quantifier ($\exists$) — recap + rule form

$\exists x P(x)$ — true if $P(x)$ holds for AT LEAST ONE x . To PROVE it, one example is enough; to DISPROVE it, you must check ALL x fail.

10.5 Negation and Quantifiers (recap, expanded)

Statement	Negation	In words
t	n	

$\neg \forall x P(x) \quad \exists x \neg P(x)$ "Not all P" = "Some x is not P"

$\neg \exists x P(x) \quad \forall x \neg P(x)$ "None are P" = "All x are not P"

Nested version (common trick question): $\neg \forall x \exists y P(x, y) \equiv \exists x \forall y \neg P(x, y)$ — negate outward-in, flipping EVERY quantifier as you pass it.

PART 11: ARGUMENTS, PREMISES, CONCLUSION & INFERENCE RULES

11.1 Argument, Premises, Conclusion

An **argument** = a set of **premises** (statements assumed true) leading to a **conclusion** (the statement being proved).

Written as:

P1

P2

...

Pn

$\therefore C$ ($\therefore$ means "therefore")

An argument is **valid** if, whenever ALL premises are true, the conclusion **MUST** also be true.

11.2 Standard Inference Rules (Propositional)

Rule name	Form	Example
Modus Ponens	$P, P \rightarrow Q \vdash Q$	"It rains" + "If rains, wet" $\vdash$ "Wet"
Modus Tollens	$\neg Q, P \rightarrow Q \vdash \neg P$	"Not wet" + "If rains, wet" $\vdash$ "Didn't rain"
Hypothetical Syllogism	$P \rightarrow Q, Q \rightarrow R \vdash P \rightarrow R$	Chain implications together
Disjunctive Syllogism	$P \vee Q, \neg P \vdash Q$	One of two options ruled out $\rightarrow$ the other holds
Simplification	$P \wedge Q \vdash P$ (or $\vdash Q$)	From an AND, extract either half

Conjunction	$P, Q \vdash P \wedge Q$	Combine two known truths into an AND
Addition	$P \vdash P \vee Q$	A true P lets you OR it with anything
Resolution	$P \vee Q, \neg P \vee R \vdash Q \vee R$	The cancelling-out rule (see Part 12)

Simplification, explicitly: if you know $P \wedge Q$ is true, you can immediately conclude P is true (and separately, Q is true) — you're just "simplifying" the AND down to one piece.

11.3 Predicate Logic's Additional Inference Rules

These let you move between quantified statements and specific instances.

Rule	Form	Meaning
Universal Instantiation (UI)	$\forall x P(x) \vdash P(c)$	If true for ALL, it's true for any SPECIFIC c
Universal Generalization (UG)	$P(c) \text{ for arbitrary/generic } c \vdash \forall x P(x)$	If true for an arbitrary (unspecified, general) element, it's true for ALL
Existential Instantiation (EI)	$\exists x P(x) \vdash P(c) \text{ for SOME new } c$	If something exists, give it a temporary name c (c must be a fresh/new symbol, not used before)
Existential Generalization (EG)	$P(c) \vdash \exists x P(x)$	If it's true for ONE specific c , then something exists satisfying it

Worked example combining these:

Premise 1: $\forall x (\text{Man}(x) \rightarrow \text{Mortal}(x))$

Premise 2: $\text{Man}(\text{Socrates})$

Step 1 (UI on Premise 1, $c=\text{Socrates}$): $\text{Man}(\text{Socrates}) \rightarrow \text{Mortal}(\text{Socrates})$

Step 2 (Modus Ponens with Premise 2): $\text{Mortal}(\text{Socrates})$

UI vs UG trick: UI goes from general $\rightarrow$ specific (safe, always works). UG goes specific $\rightarrow$ general, but ONLY if " c " was a totally arbitrary/generic placeholder, not a named specific individual like Socrates — you can't generalize "Socrates is mortal" into "everyone is mortal"!

PART 12: RESOLUTION PRINCIPLE (the exam favorite — when/why/how)

12.1 What is it?

The **Resolution Principle** is a single, powerful inference rule used to PROVE things by CONTRADICTION. Instead of directly proving the conclusion, you:

1. Assume the OPPOSITE (negation) of what you want to prove.
2. Add it to your set of premises.
3. Convert everything to **clausal form** (a big AND of ORs).
4. Keep "resolving" (cancelling out) matching opposite literals between clauses.
5. If you reach the **empty clause** $[]$ → contradiction found → original statement is PROVEN true.

12.2 Why use it?

Because it's a **single mechanical rule** (unlike juggling Modus Ponens, Modus Tollens, etc. separately) that can be automated — this is EXACTLY the algorithm Prolog uses internally to answer queries. It's why Prolog is called a "resolution-based" system.

12.3 When to apply it?

- When you need to prove a conclusion follows from a set of premises (formal proofs).
- Whenever you're asked to show HOW Prolog would internally solve a query (matches Part 4's inference explanation).
- When premises + negated conclusion can be converted into clause (OR-only) form.

12.4 How to apply it — the Resolvent

Given two clauses that share a **complementary pair** (P in one, $\neg P$ in the other), you cancel BOTH out and OR together everything else that's left. The result is called the **resolvent**.

Clause 1: $P \vee Q$

Clause 2: $\neg P \vee R$

----- (P and $\neg P$ cancel)

Resolvent: $Q \vee R$

12.5 Full Steps of the Resolution Principle

1. **Convert all premises (and the negated conclusion) into clausal form** — each sentence becomes a disjunction (OR) of literals, and the whole set is implicitly AND-ed together.
2. **Negate the conclusion** and add it to the clause set (proof by contradiction setup).
3. **Find two clauses with a complementary literal pair** (P in one clause, $\neg P$ in the other).
4. **Resolve them** → produce the resolvent (cancel $P/\neg P$, OR the rest together).
5. **Add the resolvent** to the clause set.
6. **Repeat** steps 3–5, using new and old clauses, until either:
 - You reach the **empty clause** $\square$ → contradiction found → original conclusion is **PROVEN**.
 - No more resolutions are possible → conclusion **cannot be proven** from the premises.

12.6 Clausal Form (how to convert)

To turn any FOL/propositional sentence into clausal form:

1. Eliminate $\leftrightarrow$ using $P \leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$
2. Eliminate $\rightarrow$ using $P \rightarrow Q \equiv \neg P \vee Q$
3. Push $\neg$ inward using De Morgan's + double negation
4. Standardize variable names (predicate logic) so no two clauses share a variable name
5. Move quantifiers out (predicate logic), then drop $\forall$ (implicit) and replace $\exists$ variables with functions (Skolemization — usually simplified in intro courses)
6. Distribute $\wedge$ over $\vee$ until you get a set of clauses (each clause = disjunction of literals)

12.7 Worked Resolution Example

Premises:

1. $P \rightarrow Q$
2. $Q \rightarrow R$ **Prove:** $P \rightarrow R$

Step 1 — Convert to clauses:

1. $\neg P \vee Q$
2. $\neg Q \vee R$
3. Negate conclusion ($P \rightarrow R$ becomes $\neg(P \rightarrow R) \equiv P \wedge \neg R$), split into two clauses: P and $\neg R$

Step 2 — Resolve:

- Resolve clause 1 ($\neg P \vee Q$) with clause 3a (P) → cancel $P/\neg P$ → resolvent: Q
- Resolve Q with clause 2 ($\neg Q \vee R$) → cancel $Q/\neg Q$ → resolvent: R
- Resolve R with clause 3b ($\neg R$) → cancel $R/\neg R$ → resolvent: $\square$ (**empty clause!**)

Conclusion: Empty clause reached $\rightarrow$ contradiction found $\rightarrow$ **P $\rightarrow$ R is PROVEN. ✓**

This is the exact same mechanism from Part 3.7 (empty clause) and Part 4.2 (how Prolog solves a query) — Prolog queries ARE resolution proofs running automatically.

PART 13: PROLOG CODE PRACTICE — FIBONACCI, FACTORIAL & COMMON QUESTION PATTERNS

13.1 Fibonacci in Prolog

```
fib(0, 0).           % base case 1
fib(1, 1).           % base case 2
fib(N, F) :-
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib(N1, F1),
    fib(N2, F2),
    F is F1 + F2.
```

Trace for `fib(4, F)`:

```
fib(4,F) needs fib(3,F1) and fib(2,F2)
fib(3,F1) needs fib(2,F1a) and fib(1,F1b=1)
fib(2,F1a) needs fib(1,=1) and fib(0,=0)  $\rightarrow$  F1a = 1
 $\rightarrow$  F1 (fib(3)) = 1 + 1 = 2
fib(2,F2) needs fib(1,=1) and fib(0,=0)  $\rightarrow$  F2 = 1
F = F1 + F2 = 2 + 1 = 3
```

`?- fib(4, F).  $\rightarrow$  F = 3.` (sequence: 0,1,1,2,3,5,8,...)

Trickiest part: Fibonacci needs TWO base facts (0 and 1), unlike factorial which needs only one (0), because each step depends on the PREVIOUS TWO values, not just one.

13.2 Factorial — recap with a common variant question

"Write factorial using an accumulator (tail recursion)" — a frequently asked twist:

```
factorial(N, F) :- factorial(N, 1, F).    % helper call, Acc starts at 1
```

```
factorial(0, Acc, Acc).           % base: return accumulated value
factorial(N, Acc, F) :-
    N > 0,
    Acc1 is Acc * N,
    N1 is N - 1,
    factorial(N1, Acc1, F).
```

This avoids the "back-substitution" of the plain version (Part 4.3) — the multiplication happens as you go DOWN, not on the way back UP. Often asked as "what's the difference between this and normal recursive factorial?" — Answer: accumulator style computes eagerly and doesn't need to unwind the recursion to get the answer; it's more memory-efficient (tail-recursive).

13.3 Common "Read This Prolog Code" Question Patterns

Pattern: List length (recursive counting)

```
mylen([], 0).
mylen([_|T], N) :- mylen(T, N1), N is N1 + 1.
```

`[_|T]` splits a list into Head (ignored, `_`) and Tail `T`. Base case: empty list has length 0.

Pattern: Member check

```
mymember(X, [X|_]).
mymember(X, [_|T]) :- mymember(X, T).
```

Either `X` is the head of the list, OR (recursively) `X` is a member of the tail.

Pattern: Sum of a list

```
sumlist([], 0).
sumlist([H|T], S) :- sumlist(T, S1), S is S1 + H.
```

Pattern: reverse-tracing a "what will this print" question When given code and asked "what does `?- query.` output?", ALWAYS:

1. Identify which fact/rule head matches the query first (Prolog tries clauses top-to-bottom).
2. Substitute variables as they get bound.
3. Follow the body left-to-right, AND-ing each goal.
4. If a goal fails, backtrack to the last choice point (see Part 7) and try the next alternative.

5. Stop when either every goal succeeds (empty clause reached → answer printed) or all options are exhausted (`false.`).

Pattern: Negation-as-failure trap

```
odd(X) :- \+ even(X).
```

```
even(0).
```

```
even(X) :- X > 0, X2 is X - 2, even(X2).
```

`\+ Goal` succeeds if Goal CANNOT be proven — this is "negation as failure", NOT the same as logical negation ($\neg$) from Part 1–2! It just means Prolog tried and failed, which can behave oddly with unbound variables — a classic trick-question topic ("why does `\+ even(X)` misbehave if X isn't bound yet?" — because Prolog can't search infinite unbound possibilities).